<!--POO-->

Tema de arquitectura de software(ARQUITECTURA EN CAPAZ) {
Tema patrón de diseño(SINGLETON & FACTORY METHOD)}

<Por="Dilan Bohorquez"/>

<Por="Alexandra Rodriguez"/>

<FICHA="3315796"/>



Contenidos

01 Definición.

02 Caracteristicas.

03 Diagrama.

04 Ventajas y desventajas

05 Tecnologías relacionadas

06 Demostración

07 Conclusión

08 situación en la que conviene utilizar

Definición ¿qué es? y ¿para qué sirve? {

Arquitectura en capas

¿Qué es? Es una forma de organizar un programa dividiéndolo en capas, una encima de otra. Cada capa tiene una tarea propia (mostrar pantallas, aplicar las reglas del negocio, manejar los datos) y solo se comunica con la capa vecina.

¿Para qué sirve? Sirve para tener el código ordenado, hacer cambios en una capa sin dañar las demás, probar cada parte por separado y trabajar en equipo sin estorbarse.

Singleton

¿Qué es? Es un patrón de diseño que hace que una clase tenga una sola instancia (un solo objeto) en todo el programa, y que todos accedan a ese mismo objeto.

¿Para qué sirve? Sirve para compartir un recurso único y evitar duplicados, por ejemplo la conexión a la base de datos, la configuración de la aplicación o el registro de errores (logger).

}

Caracteristicas:

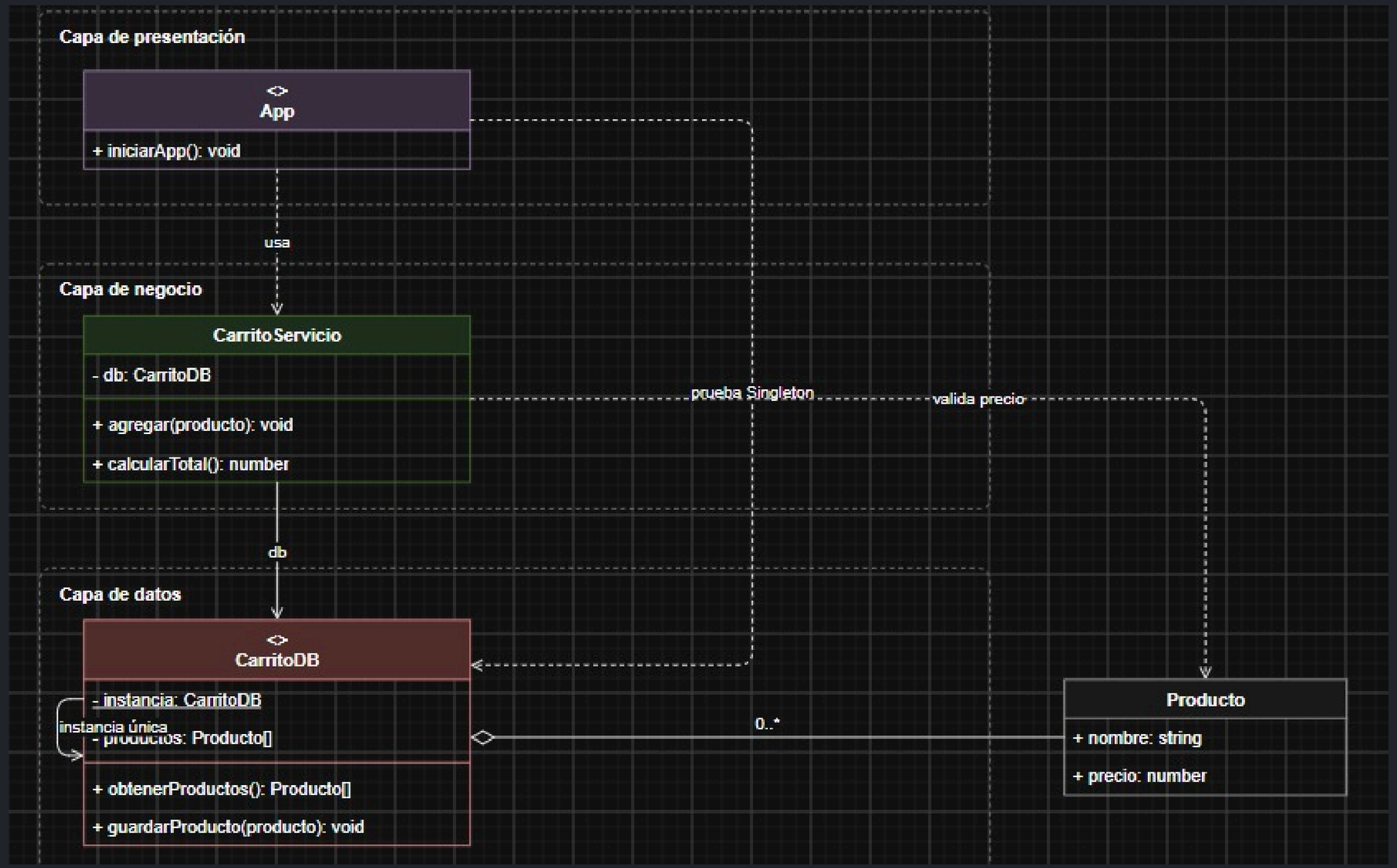
Características

1. Independencia jerárquica: Cada capa tiene una responsabilidad única y bien definida dentro del flujo de la aplicación.
2. Comunicación unidireccional: Una capa superior puede invocar a la capa inmediatamente inferior, pero la capa inferior no debe conocer los detalles de implementación de la superior.
3. Modularidad: Facilita el mantenimiento, permitiendo escalar o modificar una capa (por ejemplo, cambiar la base de datos o la interfaz visual) sin romper el resto del sistema.
4. Abstracción: Oculta los detalles complejos internos exponiendo únicamente interfaces o contratos claros.

¿Como funciona? y ¿cómo se organiza? {

// Organización de la Arquitectura en Capas:
1. Presentación (UI): Captura datos y muestra la interfaz al usuario.
2. Negocio (Lógica): Procesa las reglas, cálculos y validaciones del sistema.
3. Datos (Persistencia): Conecta con la base de datos para guardar y consultar información.
-> Cada capa funciona de forma independiente y se comunica solo con su nivel adyacent }

Diagrama: representación GRÁFICA y conexiones {



}

Ventajas{

01 Mantenimiento sencillo: Permite encontrar y corregir errores rápido al tener el código separado por responsabilidades.

02 Trabajo en equipo: Varios desarrolladores pueden trabajar al tiempo en diferentes capas (ej. frontend y backend) sin pisarse.

03 Reutilización de código: La lógica de negocio o de base de datos se puede usar en diferentes partes sin duplicarla.

04 Alta escalabilidad: Permite cambiar o mejorar una capa (como la base de datos) sin afectar a los demás.

}

Desventajas{

01 Sobrecarga de rendimiento: El paso obligatorio de datos entre capas puede agregar un pequeño retardo.

02 Complejidad inicial: Requiere más tiempo de planificación y configuración que un proyecto simple o monolítico.

03 Rigidez en cambios pequeños: Un cambio simple a veces exige tocar archivos en todas las capas.

04 Redundancia de código: Puede generar clases o funciones innecesarias solo para pasar datos de una capa a otra

}

Tecnologías relacionadas {

- **Presentación: React, HTML, CSS, JavaScript**
- **Lógica de negocio: clases en Java, Spring Boot, Node.js, Laravel, Django**
- **Acceso a datos: Hibernate/JPA, Sequelize**
- **Base de datos: MySQL**
- **Nube: AWS, Azure o Google Cloud para alojar cada capa**

Conclusión

- **Conviene en aplicaciones web y empresariales, sistemas grandes que crecen y proyectos con varios desarrolladores.**
- **No conviene en aplicaciones muy pequeñas (se vuelve complicado sin necesidad) ni cuando se necesita rendimiento extremo.**

}

Demostración {

¿Qué hicimos en este código?

- Arquitectura en Capas: Dividimos el sistema en tres partes organizadas: Presentación (simula la interfaz), Negocio (aplica las reglas y validaciones) y Datos (maneja la información).
- Patrón Singleton: En la capa de datos (CarritoDB), usamos una validación para garantizar que existe una sola instancia del carrito en toda la aplicación, evitando que se duplique la información al interactuar con el sistema.

app.js > iniciarApp > precio

```
1   // 1. CAPA DE DATOS (Singleton)
2   class CarritoDB {
3       constructor() {
4           if (CarritoDB.instancia) {
5               return CarritoDB.instancia;
6           }
7           this.productos = [];
8           CarritoDB.instancia = this;
9       }
10
11      obtenerProductos() {
12          return this.productos;
13      }
14
15      guardarProducto(producto) {
16          this.productos.push(producto);
17      }
18  }
19
20  // 2. CAPA DE NEGOCIO (Lógica)
21  class CarritoServicio {
22      constructor() {
23          this.db = new CarritoDB();
24      }
25
26      agregar(producto) {
27          if (producto.precio <= 0) {
28              throw new Error("El precio no es válido");
29          }
30          this.db.guardarProducto(producto);
31          console.log(`[Negocio] Agregado: ${producto.nombre}`);
32      }
```

Problemas Salida Terminal ··· bash + ···

```
Dilan@DESKTOP-DQ982PK MINGW64 ~/Documents/GitHub/Arqu
itectura-en-capas/presentacionPoo (main)
$ node app.js
=== TIENDA DIYEH RACING STORE ===
[Negocio] Agregado: Exosto Pulsar NS 200
[Negocio] Agregado: Pastillas de freno Duke NG
[Negocio] Agregado: Pastillas de freno Duke clasica
Total de productos en el carrito único: 3
[Presentación] Total a pagar: $285000 COP
```

}

Conclusión {

Singleton

Conviene usarlo cuando el sistema necesita una sola instancia de una clase que todos compartan. Es ideal para la conexión a la base de datos, la configuración general de la aplicación, el registro de errores (logger) o el caché, porque evita duplicar recursos y mantiene los datos consistentes.

Factory Method

Conviene usarlo cuando el programa debe crear objetos de distintos tipos y no se sabe de antemano cuál se necesitará. Permite agregar nuevos tipos sin cambiar el código que ya funciona, lo que hace el sistema más flexible y fácil de mantener.

}

Situación en la que conviene utilizar{

¿Cuándo usar cada tema?

Tema	Se usa cuando...	Ejemplo
Arquitectura en capas	Se necesita organizar toda la aplicación	Tienda en línea con React, Java y MySQL
Singleton	Se necesita una única instancia compartida	Conexión a la base de datos
Factory Method	Se necesita crear objetos de distintos tipos	Pagos con tarjeta, PSE o billetera

}

<!--POO-->

Gracias {

<Por="Dilan Bohorquez"/>

<Por="Alexandra Rodriguez"/>

<FICHA="3315796"/>

}